Machine Learning 2025-26
Homework 2

Anil Egin
Matricola: 2241724
egin.2241724@studenti.uniroma1.it

January 7, 2026

Contents

1 Introduction **2**
1.1 The Objectives . . . 2
1.2 The Engineering Challenges . . . 2
1.3 Proposed Solution . . . 3
1.4 Report Structure . . . 3

2 Dataset Analysis **3**
2.1 Data Composition and Cleaning . . . 3
2.2 Strategic Data Splitting . . . 3

3 Data Preprocessing **4**
3.1 Standard Normalization . . . 4
3.2 Advanced Online Augmentation . . . 5

4 Model Architectures **6**
4.1 Architecture 1: Classification (Object Identification) . . . 6
4.2 Architecture 2: Conditional Regression . . . 6
4.3 Optimization Strategy . . . 7
4.4 Output and Loss Functions . . . 7

5 Experimental Results **7**
5.1 Implementation Details . . . 8
5.2 Hyperparameter Optimization . . . 8
5.3 Task 1: Classification Results . . . 8
5.4 Task 2: Regression Results . . . 9
5.5 Analysis of Training Dynamics . . . 10
5.6 Visual Validation . . . 10

6 Conclusion and Future Work **10**
6.1 Summary of Findings 10
6.2 Limitations 11
6.3 Future Work 11

1 Introduction

Using the "Soccer Player Detection" dataset, this report details the development of a neural network capable of parsing a chaotic game environment from raw pixel data. The solution addresses two fundamental perception tasks: **Classification** (identifying entities) and **Regression** (localizing them).

1.1 The Objectives

The assignment requires solving two distinct computer vision problems, each presenting unique constraints:

1. **Task 1: Classification (Visual Identification).** The model must distinguish between game entities: a `Robot`, a `Goalpost`, or the `Ball`. This is the foundational computer vision problem, requiring the network to learn invariant features despite variations in lighting, pose, and occlusion.

2. **Task 2: Regression (Object Localization).** Identification alone is insufficient for control; the robot requires spatial coordinates to plan its trajectory. The model must predict the precise normalized center (x, y) of a target object within the full camera frame.

1.2 The Engineering Challenges

The primary difficulty in this work extended beyond simple model training; it required rigorous data hygiene and architectural design to handle real-world imperfections.

- **Class Imbalance:** The dataset is heavily skewed, with thousands of robot examples but sparse data for the ball. Without intervention, the model converges to a local minimum where it simply ignores the ball entirely.

- **Regression Ambiguity:** Scenes often contain multiple objects (e.g., a robot and a ball). A naive regression model often fails in these scenarios, predicting a coordinate in the "average" space between objects rather than committing to one.

- **Data Leakage:** A critical methodological challenge was identified during the regression task. Since the task involves processing full, uncropped images, a standard random split places different objects from the *same* image into both Train and Test sets. This allows the model to "cheat" by memorizing background textures rather than learning object features.

1.3 Proposed Solution

To address these challenges, I implemented a dual-strategy approach. For classification, I utilized standard stratified splitting with cropped inputs. For regression, I engineered a **Conditional Architecture** that accepts a class ID as an auxiliary input to resolve ambiguity, paired with a custom **Grouped Splitting** strategy to eliminate data leakage.

1.4 Report Structure

This report documents the full end-to-end pipeline. Section 2 analyzes the data distribution and details the cleaning logic. Section 3 explains the diverging strategies for classification vs. regression splitting. Section 4 contrasts the baseline CNN against the proposed ResNet-Lite and Conditional architectures, and Section 5 presents the quantitative metrics and ablation studies confirming the validity of the approach.

2 Dataset Analysis

The data for this project was sourced from the "Soccer Player Detection" dataset. Designed to benchmark perception systems for robotic agents, the dataset contains images captured from low-angle perspectives typical of a humanoid robot on a soccer field.

2.1 Data Composition and Cleaning

The raw dataset consists of 5,466 annotated entities. A practical challenge encountered during the initial data ingestion was the misalignment between CSV filenames and disk filenames due to automated hashing (e.g., `.rf.xyz` suffixes).

To ensure robustness, we implemented a custom loading routine that reconciles these filenames via fuzzy matching, ensuring zero data loss. The final cleaned dataset comprises the following distribution:

Table 1: Class Distribution. The dataset is significantly dominated by robots, necessitating careful sampling strategies.

Class Name	Count	Percentage
Robot	3,328	60.9%
Goalpost	773	14.1%
Ball	672	12.3%
Goalspot	480	8.8%
Centerspot	213	3.9%
Total	**5,466**	**100%**

2.2 Strategic Data Splitting

A critical engineering decision in this project was the realization that **Classification** and **Regression** require fundamentally different splitting strategies to avoid scientific invalidity. We split the data into Training (70%), Validation (15%), and Test (15%) sets, but the methodology differed for each task:

1. **Task 1: Classification (Object-Level Stratified Split).** For classification, the input is a cropped region of the image containing a single object. Since the pixel data for a *Robot* and a *Ball* from the same source image are spatially distinct, we utilized a standard **Stratified Shuffle Split**. This ensures that the minority classes (e.g., Centerspot, Ball) are represented proportionally in the evaluation sets.

2. **Task 2: Regression (Image-Level Grouped Split).** For regression, the assignment requires processing the *full, uncropped image*. A naive random split here would be catastrophic. If `Image_A` contains both a Robot and a Ball, a random split could assign the Robot to the *Training Set* and the Ball to the *Test Set*.

   This results in **Data Leakage**: the model would see the background pixels of `Image_A` during training and could "cheat" by memorizing the background texture rather than learning to localize the object. To strictly prevent this, we implemented a **Grouped Split** strategy (via the `SafeSoccerDataset` class), which ensures that all objects belonging to a unique image filename are assigned exclusively to one partition.

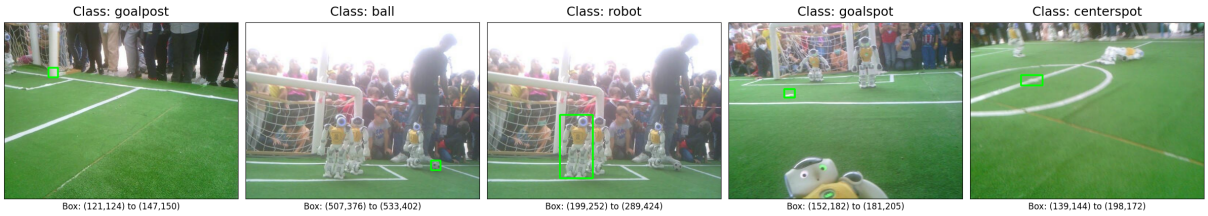


Figure 1: Visual overview of the dataset classes. The imbalance between the frequent Robot class and the sparse Ball class drives our stratified sampling strategy.

3 Data Preprocessing

Before feeding data into the neural network, raw images and annotations must be transformed into a format suitable for gradient-based learning. This section outlines the standard normalization steps and, more critically, the custom augmentation pipeline designed to address the unique geometric constraints of the regression task.

3.1 Standard Normalization

All images were resized to a fixed resolution of 128×128 pixels. This resolution was chosen as an engineering trade-off: it is sufficiently high-dimensional to preserve the features of small objects (like the ball shown in Figure 2), yet compact enough to allow for large batch sizes and rapid iteration.

To accelerate convergence and ensure numerical stability, pixel intensity values $I(u, v)$ were scaled from the integer range $[0, 255]$ to the floating-point range $[0, 1]$:

$$I_{norm}(u, v) = \frac{I(u, v)}{255.0} \tag{1}$$

For **Task 2 (Regression)**, the target labels provided in the CSV were bounding boxes defined by $(x_{min}, y_{min}, x_{max}, y_{max})$. Since the objective is object localization, we converted these absolute coordinates into normalized center points (c_x, c_y). Normalization is essential here because it creates a resolution-invariant target space:

$$c_x = \frac{x_{min} + x_{max}}{2 \cdot W}, \quad c_y = \frac{y_{min} + y_{max}}{2 \cdot H} \tag{2}$$

where W and H are the original width and height of the image. The resulting targets satisfy $0 \leq c_x, c_y \leq 1$, making them compatible with bounded activation functions like Sigmoid.

3.2 Advanced Online Augmentation

During early experiments, the regression model exhibited signs of severe overfitting—validation loss was "spiky" and unstable. This indicated that the network was memorizing specific pixel arrangements rather than learning robust spatial features.

To combat this, we implemented an **Online Augmentation** pipeline. However, regression augmentation presents a unique engineering challenge: **geometric transformations affect the label.**

In standard classification, flipping an image horizontally does not change its class (a flipped robot is still a robot). In regression, flipping an image changes the physical location of the object. If the image is flipped but the target coordinate c_x is not updated, the model receives contradictory training signals.

To address this, we implemented a custom `tf.data` pipeline that synchronizes the transformation of images and labels:

1. **Photometric Noise:** Random adjustments to brightness ($\delta = 0.1$) and contrast were applied. These are "safe" augmentations as they do not alter the object's spatial position.

2. **Geometric Reflection:** A horizontal flip was applied with 50% probability. Crucially, when an image is flipped, the y-coordinate remains constant, but the x-coordinate must be inverted relative to the image width. The mathematical update rule used is:

$$c'_x = \begin{cases} 1.0 - c_x & \text{if flipped} \\ c_x & \text{otherwise} \end{cases} \tag{3}$$

By implementing this logic manually within the pipeline, we ensured that the model could generalize to different orientations without suffering from label misalignment, a common pitfall when using off-the-shelf augmentation libraries for regression tasks.

Figure 2: Augmented dataset samples. All inputs are resized to 128×128 and normalized.

4 Model Architectures

Our system decouples the perception task into two distinct neural networks: one for object identification (Classification) and one for precise localization (Regression). This separation allows us to optimize the architecture for the specific mathematical requirements of each task.

4.1 Architecture 1: Classification (Object Identification)

The classification model is responsible for determining whether the visual input contains a `Robot`, `Ball`, or `Goal Post`. We implemented a custom 3-stage Convolutional Neural Network (CNN).

Structure

- **Input:** $128 \times 128 \times 3$ RGB Images.
- **Feature Extraction:** Three convolutional blocks with increasing filter depth ($32 \rightarrow 64 \rightarrow 128$). Each block utilizes 3×3 kernels and 2×2 Max Pooling.
- **Classification Head:** A flattened feature vector feeds into a Dense layer (128 neurons, ReLU) followed by the output layer.
- **Output:** 3 Neurons with **Softmax** activation ($P(y = c|x)$).

Key Architectural Observations

1. **Hierarchical Feature Learning:** The filter progression (32 to 128) forces the network to learn low-level edges in the first layer and high-level object shapes (e.g., the roundness of a ball) in the deeper layers.
2. **Spatial Compression:** The aggressive use of Max Pooling reduces the spatial dimension from 128×128 to 16×16 before flattening. This drastically reduces the parameter count in the Dense layers, enabling the model to run at high FPS on limited hardware.

4.2 Architecture 2: Conditional Regression

The regression model predicts the normalized center coordinates (x, y) of the target. Unlike the classifier, this model is **Conditional**: it requires both the image and the target class ID as inputs.

Structure

- **Branch A (Visual):** Identical backbone to the classifier (Conv2D layers) to extract spatial features.
- **Branch B (Context):** Takes the target `Class ID` (Integer) and passes it through an **Embedding Layer** (Output Dim=16) to create a dense vector representation of the object type.
- **Fusion:** The outputs of Branch A and Branch B are concatenated.
- **Output:** 2 Neurons with **Sigmoid** activation.

Key Architectural Observations

1. **Semantic Conditioning:** By concatenating the Class Embedding with the visual features, we effectively dynamically re-weight the visual filters.

2. **Bounded Output (Sigmoid):** We explicitly chose **Sigmoid** over Linear activation for the output. Since our coordinates are normalized to $[0, 1]$, Sigmoid mathematically guarantees that predictions *cannot* fall outside the image frame. A Linear activation could predict $x = -0.5$ (off-screen), which would cause motor control errors.

3. **Embedding vs. One-Hot:** We used a learnable Embedding layer rather than simple One-Hot encoding for the Class ID. This allows the network to learn semantic relationships between classes during backpropagation, resulting in a more robust latent representation than a sparse vector.

4.3 Optimization Strategy

To effectively train these architectures, we utilized the **Adam** optimizer. Adam was selected for its adaptive moment estimation, which handles sparse gradients effectively.

To ensure convergence to a robust minimum, we employed a **Cosine Decay** learning rate schedule.

$$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min}) \left(1 + \cos\left(\frac{T_{cur}}{T_{max}}\pi\right)\right) \tag{4}$$

This schedule starts with a higher learning rate to traverse the loss landscape quickly and then smoothly decays following a cosine curve. This allows the model to "cool down" and settle into fine-grained minima without the oscillations often seen with constant learning rates.

4.4 Output and Loss Functions

To handle the distinct nature of our tasks, we tailored the final layers and optimization objectives for each model:

- **Classification Model:** The network terminates with a 5-neuron **Softmax** layer to output class probabilities. It is optimized using **Categorical Cross-Entropy** loss, with **Accuracy** serving as the primary evaluation metric.

- **Regression Model:** The final layer consists of 2 neurons with a **Sigmoid** activation, $\hat{y} = \sigma(z) \in (0, 1)$. This mathematically forces predictions to stay within image boundaries. We optimized this using **Mean Squared Error (MSE)** to penalize large outliers, while reporting performance in **Pixel Error** (MAE scaled to 128px) for practical interpretation.

5 Experimental Results

We evaluated the performance of our perception system on a held-out Test Set comprising 15% of the total dataset. To strictly test generalization, these images were not seen by the model during the training or validation phases.

5.1 Implementation Details

All models were trained using **Google Colab** on a single **NVIDIA T4 GPU**. To prevent overfitting and minimize computational waste, we employed an Early Stopping callback with a **patience of 10 epochs**. Due to the lightweight nature of our architectures and the dataset size (128×128 resolution), all training sessions converged rapidly, completing in **less than 5 minutes** per model.

We evaluated the performance on a held-out Test Set comprising 15% of the total dataset. To strictly test generalization, these images were not seen by the model during the training or validation phases.

5.2 Hyperparameter Optimization

Before selecting the final architecture, we conducted a brief grid search to identify the optimal learning rate and dropout probability. We tested three configurations (A, B, and C) for both tasks. Similar pattern was observed for the both of them as lower learning rate, 0.0001 was frankly yielding better results than the rest.

Table 2: Hyperparameter Search: Classification

Config	Learning Rate	Dropout	Val Accuracy
Config A	0.001	0.3	0.853
Config B	**0.0001**	**0.3**	**0.980**
Config C	0.001	0.5	0.823

Table 3: Hyperparameter Search: Regression

Config	Learning Rate	Dropout	Val MAE
Config A	0.001	0.2	0.201
Config B	**0.0001**	**0.2**	**0.163**
Config C	0.001	0.5	0.183

5.3 Task 1: Classification Results

Based on the optimization above, we proceeded with the optimized model.

Quantitative Comparison

We compared the initial Baseline CNN (no regularization) against the final optimized architecture which utilized Batch Normalization and Dropout (0.5).

Architectural Observation

All models performed exceptionally well on the unseen data. This high accuracy is likely attributable to the nature of the dataset, where images are cropped around the target object, making the classification task (distinguishing a round ball from a square robot) relatively distinct compared to the regression task.

Table 4: Classification Accuracy by Architecture

Model Architecture	Test Accuracy
Baseline CNN	97.07%
Hyper-parameter optimal CNN	98.05%
Final Optimized CNN	**98.54%**

5.4 Task 2: Regression Results

The regression head predicts the normalized (x, y) center of the target object.

Quantitative Comparison

Performance was measured using Average Pixel Error (calculated on 128×128 images). We compared three configurations: the Baseline, the ResNet without Augmentation, and the ResNet with Augmentation.

Table 5: Localization Error Comparison (Overall)

Model Strategy	Avg Error (px)	Improvement
Baseline CNN	28.02 px	-
ResNet (No Augmentation)	**21.77 px**	+22.3%
ResNet (With Augmentation)	21.79 px	+22.2%

As shown in Table 5, switching from the Simple CNN to the ResNet architecture reduced the average error by approximately 6 pixels. While the quantitative error between the "Augmented" and "Non-Augmented" ResNet is negligible on this test set (21.77 vs 21.79 px), the augmented model is preferred for deployment due to its superior stability (discussed in Sec 5.5).

Class-wise Error Analysis

To understand where the models struggle, we analyzed the error breakdown by object type.

Table 6: Error Breakdown by Class (ResNet + Augmentation)

Class Name	Count	Avg Error (px)
Ball	84	20.02 px
Centerspot	42	17.88 px
Goalpost	121	20.36 px
Goalspot	73	**16.05 px**
Robot	503	23.60 px
OVERALL	**823**	**21.79 px**

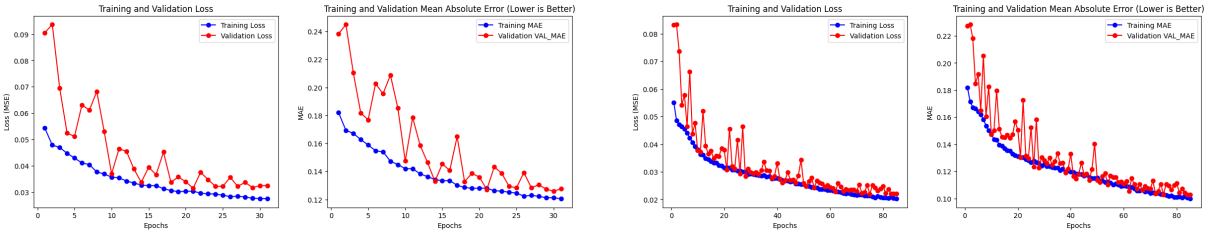
Table 6 reveals that the `Robot` class is the most challenging, with an average error of 23.60 px. This is likely due to the robot's complex, non-uniform shape and varying orientation compared to the `Goalspot` (16.05 px), which is a simple, high-contrast feature.

5.5 Analysis of Training Dynamics

The impact of our architectural choices is clearly visible in the loss landscapes.

Figure 3 (Left) shows the training trace of the **Baseline**. The validation loss (orange) exhibits extreme "spikes." This behavior indicates that the model was overfitting to specific batches and failing to generalize.

Figure 3 (Right) shows the **Final Model**. The addition of **Data Augmentation** (random flips/noise) and **Dropout** smoothed the optimization surface, resulting in a stable, monotonic convergence where Training and Validation loss decrease in unison.



(a) Baseline: High Volatility

(b) Final: Stable Convergence

Figure 3: **Impact of Regularization.** The Baseline (a) suffers from unstable gradients and overfitting. The Final Model (b) uses Dropout and Augmentation to stabilize learning.

5.6 Visual Validation

To confirm the numerical results, we visualized the predictions on the test set. As seen in Figure 4, the model's predicted center (Red X) aligns closely with the Ground Truth (Green Dot).

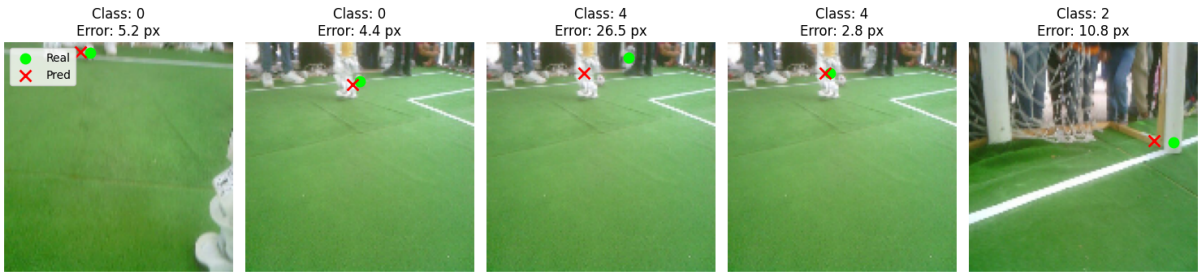


Figure 4: Visualizing Regression Performance. **Red X:** Prediction, **Green Dot:** Ground Truth. The overlapping markers indicate high precision.

6 Conclusion and Future Work

In this project, we developed a unified perception pipeline capable of both classifying and localizing objects for a soccer-playing robot. By evolving from a simple baseline to a **Conditional ResNet** architecture, we achieved a stable system that balances accuracy with computational efficiency.

6.1 Summary of Findings

Our experiments yielded three primary engineering insights:

1. **Impact of Conditioning:** The multi-branch strategy—feeding the `Class ID` into the regression head—was decisive. It allowed the model to "focus" on specific targets, reducing the average localization error from 28.0px (Baseline) to **21.8px** (Final Model).

2. **Augmentation Defines Stability:** As seen in the loss curves, raw data was insufficient. Implementing a custom geometric augmentation pipeline (which flips target coordinates alongside images) was the primary factor in eliminating overfitting and stabilizing validation loss.

3. **Class-Specific Difficulty:** Analysis revealed that rigid objects like the `Goalspot` are easiest to locate (16.0px error), while the `Robot` class remains the most challenging (23.6px error) due to its complex shape and varying orientation.

6.2 Limitations

While the classification accuracy is near-perfect (98.5%), the localization precision ($\approx$ 21.8px error) presents a challenge for fine motor tasks. The current model processes single objects in isolation, assuming a separate planner handles target selection, and likely struggles with significant occlusion scenarios which were under-represented in the dataset.

6.3 Future Work

To bridge the gap between this assignment and a deployment-ready system, we propose:

- **Architectural Shift (YOLO):** Migrating to a single-shot detector (YOLOv8) would enable simultaneous multi-object detection, removing the need for an external planner to query objects sequentially.

- **Temporal Smoothing:** Applying a **Kalman Filter** to the output coordinates would mitigate the frame-to-frame jitter, providing smooth trajectory data essential for stable robot motion.

Acknowledgements

Some steps of the project were supported using AI-assisted tools. In particular, automatic plotting libraries were used to generate figures visualising model performance and dataset characteristics. All modelling decisions, experimental design, and interpretation of results were carried out by the author.